

CS184/284A Spring 2026

Homework 1 Write-Up

Johnson Huang

Link to webpage: <https://cal-cs184-student.github.io/hw-webpages-Coffeepot223/>

Link to GitHub repository: <https://github.com/cal-cs184-student/hw1-rasterizer-jonjon>

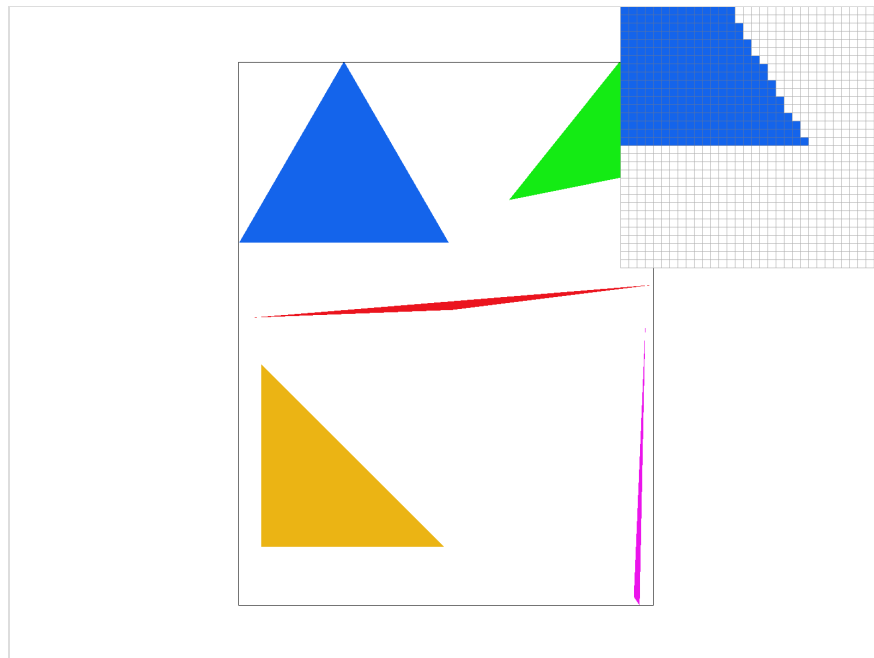
Overview

In this homework, I implemented a rasterizer capable of drawing triangles with single colors, interpolated colors, and textures using various different sampling techniques. This project taught me the fundamental trade-offs between computational speed, memory usage, and visual quality in computer graphics.

Task 1: Drawing Single-Color Triangles

For the rasterizations, I implemented the standard point-in-triangle test. I first calculate the bounding box of the triangle by finding the minimum and maximum X and Y coordinates of the three vertices. This allows me to restrict the search area significantly compared to checking every single pixel in the framebuffer. I also ensured that these coordinates were clamped to the screen width and height, so the program doesn't crash if

a triangle exceeds the bounds. Once I have the loop set up, I iterate through every single pixel inside the box. For each pixel, I sampled at the center $(x+0.5, y+0.5)$ and ran the edge function on all three edges of the triangle. Additionally, to handle both clockwise and counterclockwise winding orders, I just check if the results of the edge tests are either all positive or all negative. If they match, then the pixel is inside, and I can call `fill_pixel`. My algorithm is no worse than checking each sample within the bounding box because it never checks a pixel outside the rectangular bounds defined by the vertices. This gives me a runtime of $O(W \times H)$ instead of $O(\text{Screen } W \times \text{Screen } H)$.

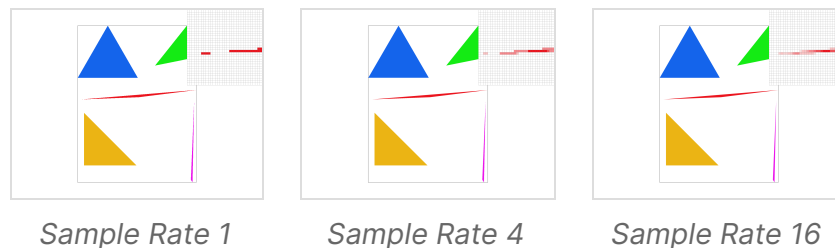


Task 2: Antialiasing by Supersampling

To implement the supersampling feature, I modified my rasterization to use a larger intermediate buffer, called the `sample_buffer`. Instead of storing one color per pixel, this new buffer now stores `sample_rate` number of colors for each pixel. I then updated `set_sample_rate` and

`set_framebuffer_target` to resize this buffer whenever the settings change. In the triangle rasterization function, I added two nested loops to iterate through a sub-pixel grid for each pixel. For each sub-pixel, I performed the point-in-triangle test at the sub-pixel's center and stored the result in the `sample_buffer`. Finally, in `resolve_to_framebuffer`, I averaged the colors of all sub-pixels within a single pixel to produce the final antialiased color.

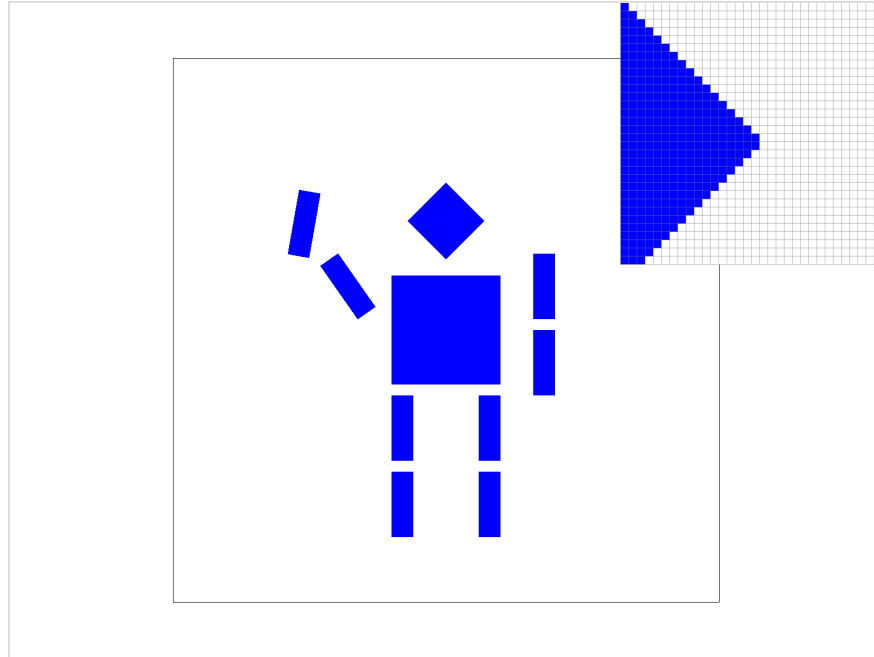
Supersampling is useful because it softens "stair-step" edges or aliasing by representing partial pixel coverage. By sampling at a higher frequency and averaging, we create smoother transitions. At a sample rate of 1, the skinny triangle corners appear disconnected because the single sample point misses the thin geometry. As the sample rate increases to 4 and 16, these edges become smoother, and the thin tips become more visible. This happens because the sub-pixels capture partial hits on the triangle, which are then averaged into various shades of the triangle's color.



Task 3: Transforms

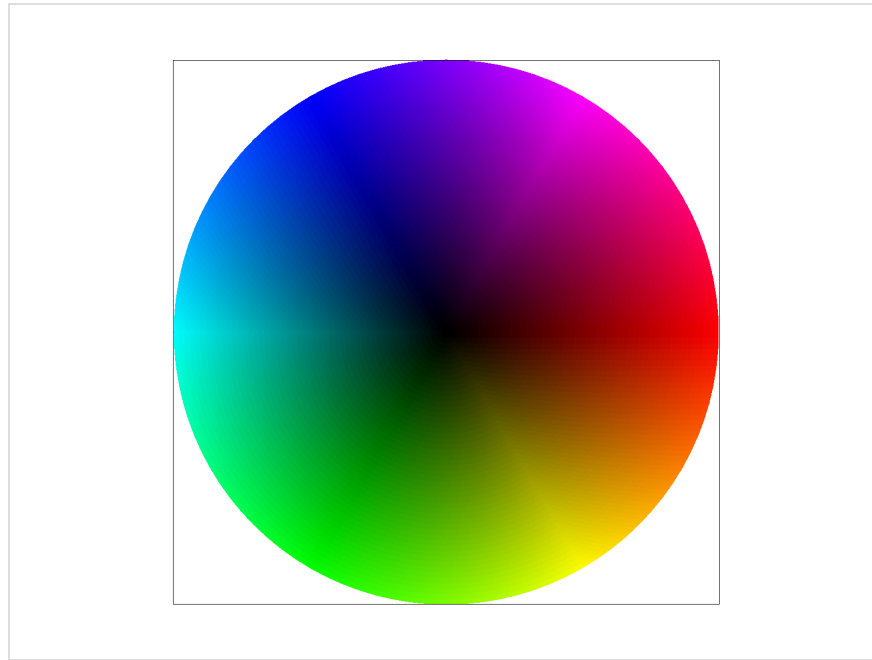
For translation, I created a matrix that shifts coordinates by dx and dy . Then, for scaling, I adjusted the diagonal elements of the matrix to grow or shrink shapes along the X and Y axes. And for rotation, I converted the input degrees into radians and then used the standard sine and cosine formulas to rotate the coordinates counterclockwise. I modified Cubeman to be completely

blue and changed its pose so it looks like it is waving with its left arm while its right arm hangs naturally at its side. To do this, I simply modified the transforms by adding rotation values to the specific arm segments and updating the fill color for all polygons to blue.



Task 4: Barycentric Coordinates

Barycentric coordinates are a coordinate system that can describe a point's position within a triangle based on its distance from the three vertices. Essentially, every point inside the triangle can be expressed as a weighted average of the three corners, where the weights always add up to 1. What I did in the implementation was to rasterize these weights for each sample point during rasterization, by calculating how much influence do the corners have over a specific pixel's coordinates. By mixing the colors of 3 vertices with these weights, I created smooth color gradients over my entire triangle.

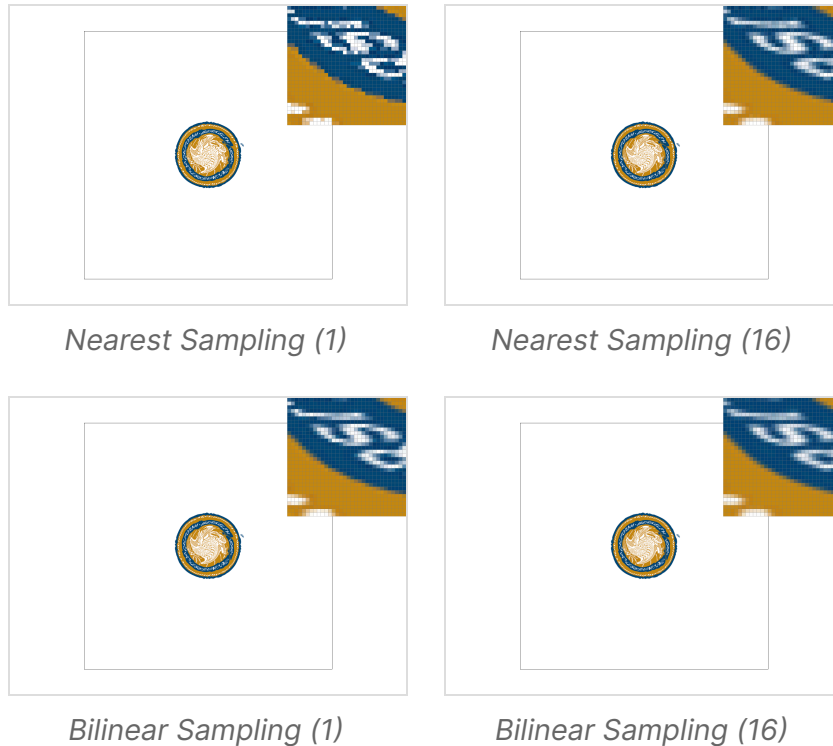


Task 5: Pixel sampling

Pixel sampling is when we take the UV coordinates of a pixel and use them to find the right color from a texture map. In my implementation, I updated the `rasterize_textured_triangle` function to calculate these UV coordinates at every sample point by interpolating the vertex using the barycentric math from Task 4. These coordinates are then used to look up the color in the texture through two different methods. Nearest neighbor sampling, where we simply round the UV coordinates to the closest integer texel, which is very fast but can look blocky when you zoom in. Bilinear sampling finds the four closest texels to the sample point and then performs a weighted average based on their distance, which creates a much smoother transition between colors at the cost of computational power.

Bilinear sampling looks better than nearest sampling in this example. Nearest sampling causes a "stair-step" look because it forcefully picks one color for the entire region, whereas bilinear sampling creates a smooth gradient between the four nearest colors. The difference

is also most dramatic at 1 sample per pixel because as we increase the sample rate to 16, supersampling helps hide some of the blockiness in Nearest, but Bilinear still provides a more natural and smooth look.



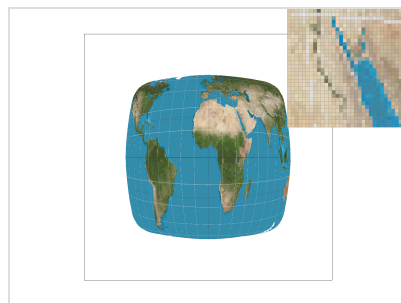
Task 6: Level Sampling with Mipmaps

Level sampling is the process of taking the UV coordinates of a pixel and determining which version of a texture from it is the most appropriate based on the pixel's footprint on the screen. To implement this, I updated `rasterize_textured_triangle` to calculate the UV coordinates for two adjacent points, $(x+1, y)$ and $(x, y+1)$, using barycentric differentials. By measuring how much these UVs change across a single pixel, I was then able to calculate the correct mipmap level. After that, I implemented three sampling methods: `L_ZERO` for the base texture, `L_NEAREST` to pick the closest, and

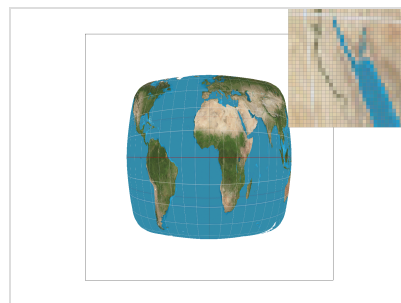
L_LINEAR, which linearly interpolates between the two closest mipmap levels.

There are also significant tradeoffs between speed, memory, and antialiasing power across these techniques. Pixel sampling prioritizes either speed or smoothness. Nearest neighbor is fast but creates blocky magnification artifacts, while Bilinear sampling is slightly slower but provides smoother transitions. Level sampling using mipmaps effectively eliminates "shimmering" in the distance but requires more memory to store pre-filtered levels. Finally, increasing the sample rate through supersampling offers the highest quality for both edges and textures, but it is the most computationally expensive.

Level sampling is the most effective at preventing aliasing. Using L_NEAREST or L_LINEAR significantly reduces the noisy artifacts seen in the far distance compared to L_ZERO. Furthermore, when combined with Bilinear pixel sampling, the result is a cleaner image that avoids both blockiness and high-frequency noise.



L_ZERO, P_NEAREST



L_ZERO, P_LINEAR



L_NEAREST, P_NEAREST



L_NEAREST, P_LINEAR